S&P GLOBAL & CRISIL CAMPUS HACKATHON 2026

AI/NLP Risk Engine with Sentiment-Driven Index Rebalancing

Turning news and social-media text into structured risk signals (sentiment, event type, tickers, impact), then testing whether those signals can drive a tactical stock index.

Sumit Reddy

Indian Institute of Technology Guwahati

sumit.reddy@iitg.ac.in

Banks get a constant stream of text but need risk signals their systems can act on

The business problem

- Headlines and posts arrive faster than analysts can read them, and they are **unstructured**, so risk and portfolio tools cannot consume them directly.
- Each item needs answers to four questions: *Is this good or bad? What kind of event is it? Which companies does it affect? How much does it matter?*
- Before a bank acts on a text signal, it needs to know whether the signal actually **predicts anything**.

Scale of this prototype

1,611 signals from tweets + news	**8,233** tweets in the backtest
4,846 labeled headlines for evaluation	**2 min** live polling interval

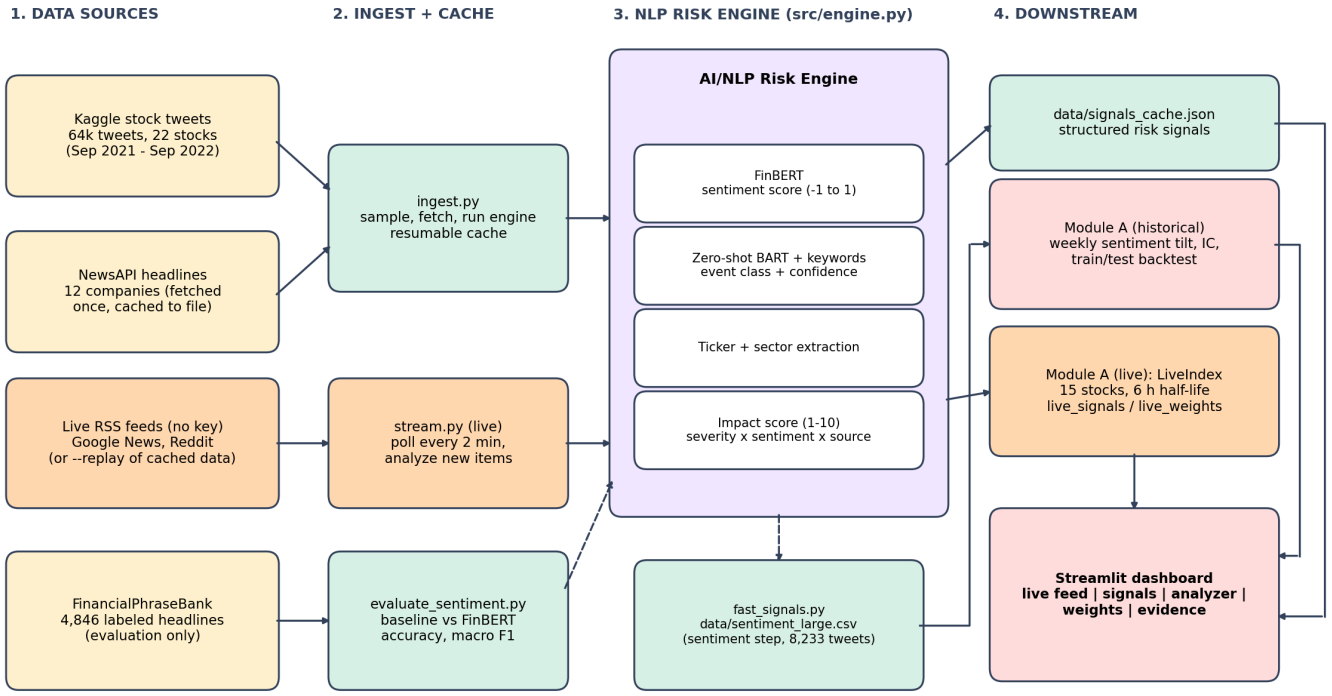
Approach: one engine with four outputs per text

Sentiment FinBERT score, −1 to +1 › **Event class** 11 types + confidence › **Tickers** $cashtags, names, sectors › **Impact** 1–10 severity score

Then use and test it

- **Module A, historical:** each week, tilt a 22-stock index toward stocks with positive tweet sentiment. Backtested with no look-ahead, 10 bps trading costs and a train/test split.
- **Module A, live:** poll Google News and Reddit every 2 minutes and update a 15-stock S&P 100 index as the signals arrive.
- **Evaluation:** score the sentiment model on 4,846 labeled headlines against a keyword baseline.

Sources → ingest & cache → NLP engine → consumers



Dashed: scripts reusing the engine's sentiment step. The historical Module A reads sentiment_large.csv (the full engine is too slow for every tweet); the live path runs the full engine on each polled item.

Batch path

Tweets and NewsAPI headlines run through `ingest.py` and the full engine into `signals_cache.json` (1,611 signals). Runs can resume after a stop.

Backtest path

`fast_signals.py` runs only the sentiment step on 8,233 tweets (zero-shot is too slow for all of them) for the weekly Module A backtest.

Live path

`stream.py` polls RSS every 2 minutes, runs the full engine on new items and updates the LiveIndex. `--replay` gives an offline demo.

All paths feed a 5-tab **Streamlit dashboard**.

Diagram: docs/architecture.png

Key modules and why each tool was chosen

Sentiment: FinBERT (`ProsusAI/finbert`)

A model trained on financial language, so words like "beats" and "downgrade" are read correctly. The score is P(positive) − P(negative). Texts are processed in batches of 32.

Events: zero-shot BART-MNLI + keyword fallback

Classifies text into 11 event types without labeled training data, using descriptive labels. When the model's confidence is below 0.6, a whole-word keyword match takes over.

Rule layers for known model gaps

A **policy rule** handles neutral-sounding burdens such as "twice the tax rate". A **per-stock rule** applies when a text names several stocks: a move word like "falls" within 3 words of a stock name sets that stock's sign.

LiveIndex: streaming Module A

Each stock's mood is the sum of sentiment × impact, and each signal fades with a 6-hour half-life. Weights are proportional to exp(0.3·z), with a z-score floor so that one headline cannot swing the whole index.

Engine output (real live signal)


```
{ "text": "Jamie Dimon Warns Economic Risks
   Are 'Shifting Below The Surface' ...
   As Wars, Sticky Inflation, And
   Debt Loom",
  "source": "news",
  "sentiment_score": -0.925,
  "event_class": "Geopolitical",
  "event_confidence": 0.56,
  "impact_score": 10.0 }
```


Stack

Python 3.13 · Hugging Face Transformers + PyTorch · pandas/numpy · yfinance · NewsAPI · RSS (Google News, Reddit) · Streamlit · scikit-learn · matplotlib

Built to run anywhere

The dashboard reads cached results, so it needs no API keys, model downloads or internet. If the models are missing, the code falls back to keyword scoring and labels the output accordingly.

FinBERT clearly beats the keyword baseline, but tweet sentiment did not beat equal weight

Sentiment on FinancialPhraseBank (4,846 headlines)

0.830 macro F1
FinBERT (accuracy 0.828)

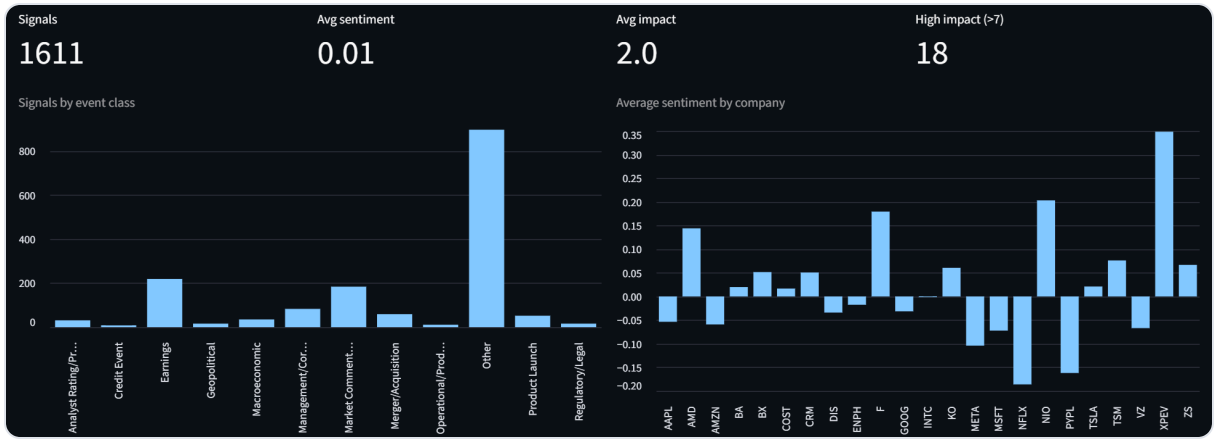
0.299 macro F1
Keyword baseline (accuracy 0.603)

Macro F1 is **2.8× the baseline's**; the baseline mostly predicted "neutral". These scores are optimistic because FinBERT was trained on this dataset.

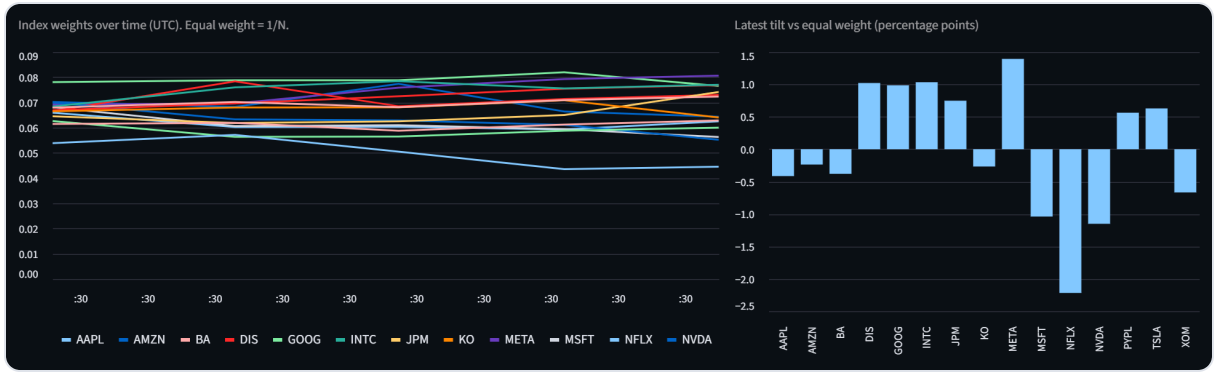
Module A backtest: 22 stocks, 54 weeks, 10 bps costs

	RANK IC (T)	EQUAL WT	TILT (NET)	LONG-SHORT
All weeks	+0.012 (0.42)	−32.2%	−33.4%	−26.7%
Train half	+0.067 (1.58)	−5.1%	−4.6%	−1.3%
Test half	−0.041 (−1.11)	−28.6%	−30.2%	−25.7%

The IC is insignificant and changes sign out of sample, so this is an honest **negative result**. The test period was a falling market for these stocks.



Risk signals tab: 1,611 signals (1,405 tweets + 206 news), event mix and average sentiment per company



Live feed tab: real run on Google News (200 signals), index weights over time and current tilt vs. equal weight

06 · DOMAIN IMPACT

Why it matters to a bank's risk and investment teams

Machine-readable risk

Free text becomes a fixed JSON record: **sentiment, event type, tickers and impact**. Monitoring, alerting and portfolio tools can consume it without anyone reading the headline.

Example: route Credit Event or Geopolitical signals with impact ≥ 7 straight to a risk desk.

Real-time early warning

The live pipeline turns public feeds into index moves **within minutes**. Signals fade over time, so the index responds to news and then relaxes back to equal weight.

The analyzer tab lets a user type a headline and watch the weights move.

Validate before you trust

Models are benchmarked against a baseline, and strategies are tested out of sample with trading costs. This showed that **tweet sentiment alone should not drive allocation**, which saves a desk from a costly false signal.

The same test framework can evaluate any new signal source.

Where a bank could use it

Credit risk

Early watch-list built from default, downgrade and bankruptcy news on borrowers

Market risk

Event-tagged news overlay that explains intraday moves in positions

Asset management

Research on sentiment tilts, with a built-in out-of-sample test

Compliance

Tracking of regulatory, legal and probe events for monitored names

Known gaps, stated openly, and what comes next

Challenges & limitations

- **Multi-stock texts** get one sentiment score plus a 3-word proximity rule. True per-company (targeted) sentiment is not solved.
- **Implied negatives** such as a tax rise without negative wording score near neutral in FinBERT. A policy rule patches this, but the rule is not validated.
- **Speed:** zero-shot BART is too slow for 64k tweets, so the backtest uses sentiment only.
- **Thin data:** about 7 tweets per stock-week over one year. Impact scores and sector maps are hand-set heuristics.
- **Live sources** are unauthenticated and rate-limited (Reddit returned HTTP 429). Reddit stands in for Twitter/X, and the live settings are unvalidated.
- The backtest uses the dataset's 22 stocks rather than S&P 100 names, and **Module B (stress testing) was not built**.

Next steps

- Targeted, per-company sentiment for multi-stock texts
- Tweet-specific model (FinTwitBERT) in a head-to-head comparison
- Use all 64k tweets and a longer, multi-regime history
- Event-level and volatility signals in addition to direction
- Module B: stress testing driven by event signals
- Evaluate live signals against realized returns

Run it: `pip install -r requirements.txt` then `streamlit run src/app.py`, and `python src/stream.py --replay` for the live demo.